Lab 02A - Version Control using Git and Repository Synchronization with GitHub

Estimated timing: 60 minutes

Prerequisites

- Completion of **Lab 01A - Agile Planning and Management using GitHub**. This lab uses the same GitHub account and the **DevOpsCoreIntroRepo** repository created in Lab 01A.
- A computer running Windows 10 or Windows 11 with internet access and permission to install software. The commands also work on macOS and Linux; notes describe the differences.
- A modern web browser.

Objectives

This lab covers the following objectives:

- Install and configure Git and Visual Studio Code
- Create a local repository and record changes by using the Git command line
- Work with branches, merges, and tags
- Synchronize a local repository with a GitHub repository
- Use Visual Studio Code to clone, commit, branch, synchronize, and resolve merge conflicts

The lab has two parts. Exercises 1 to 3 use the Git command line directly to build familiarity with the commands. Exercise 4 repeats the same workflow in Visual Studio Code, which runs the same Git commands behind its graphical interface.

Exercise 1: Install and configure Git

The exercise consists of the following tasks:

- Task 1: Install Visual Studio Code
- Task 2: Install Git
- Task 3: Configure Git
- Task 4: Verify the GitHub authentication method

Note: Visual Studio Code is installed first because the Git for Windows installer offers it as the default editor only when it is already present.

Task 1: Install Visual Studio Code

1. Start a web browser and navigate to https://code.visualstudio.com/download.
2. Select the **Windows** download (**User Installer**, **x64**) and run the downloaded file.
3. Accept the license agreement and keep the default destination folder.
4. On the **Select Additional Tasks** page, ensure that the following options are selected:
    - **Add "Open with Code" action to Windows Explorer directory context menu**
    - **Add to PATH (requires shell restart)**
5. Select **Install** and then select **Finish** to start Visual Studio Code.
6. Close Visual Studio Code.

    Note: On macOS, download the macOS build, move **Visual Studio Code** to the **Applications** folder, start it, open the Command Palette (**Shift+Command+P**), and run **Shell Command: Install 'code' command in PATH**. On Linux, install the **.deb** or **.rpm** package from the same page.

Task 2: Install Git

1. Open **Windows PowerShell** from the Start menu and run the following command to check whether Git is already installed:

    ```
    git --version
    ```

2. If the command returns a version number, such as `git version 2.56.0.windows.2`, Git is already installed. Close PowerShell and continue with **Task 3**. Otherwise, continue with the next step.
3. In the web browser, navigate to https://git-scm.com/install/windows and download **Git for Windows/x64 Setup**. On Arm based devices, download **Git for Windows/ARM64 Setup** instead.
4. Run the downloaded installer, approve the administrator prompt, and select **Next** on the license page.
5. Keep the default destination folder and the default components, and select **Next** until the **Choosing the default editor used by Git** page appears.
6. Complete the following wizard pages as described, and accept the default option on every other page:
    - **Choosing the default editor used by Git**: select **Use Visual Studio Code as Git's default editor**.
    - **Adjusting the name of the initial branch in new repositories**: select **Override the default branch name for new repositories** and enter `main`.
    - **Adjusting your PATH environment**: keep **Git from the command line and also from 3rd-party software**.
    - **Choosing HTTPS transport backend**: keep **Use the OpenSSL library**. In corporate networks that inspect TLS traffic, select **Use the native Windows Secure Channel library**.
    - **Configuring the line ending conversions**: keep **Checkout Windows-style, commit Unix-style line endings**.
    - **Choose a credential helper**: keep **Git Credential Manager**.
7. Select **Install**. When the installation completes, select **Launch Git Bash**, clear **View Release Notes**, and select **Finish**.
8. In the **Git Bash** window, run the following command to verify the installation:

```
git --version
```

Note: As an alternative to the installer, install Git with one of the following commands:

```
# Windows (PowerShell)
winget install --id Git.Git -e --source winget
# macOS (Terminal): Apple build or Homebrew
xcode-select --install
brew install git
# Ubuntu or Debian
sudo apt update && sudo apt install git -y
```

Important: The remaining steps of Exercises 1 to 3 use **Git Bash** on Windows so that the commands are identical on every platform. On macOS and Linux, use **Terminal** instead.

Task 3: Configure Git

1. In **Git Bash**, run the following commands to set the identity that Git records in every commit. Replace `<full-name>` and `<email-address>` with the name and the email address associated with the GitHub account used in Lab 01A.

```
git config --global user.name "<full-name>"
git config --global user.email "<email-address>"
```

2. Run the following commands to set the default branch name, the default editor, and the behavior of `git pull`:

```
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global pull.rebase false
```

   Note: Setting `pull.rebase` to `false` makes `git pull` merge remote changes, which is the behavior used later in this lab. Without this setting, Git on macOS and Linux stops with a message that asks how to reconcile branches that have diverged.

3. On macOS and Linux only, run the following command to normalize line endings. The Git for Windows installer already configured the equivalent setting on Windows.

```
git config --global core.autocrlf input
```

4. Run the following command to review the settings and the configuration file that defines each one:

```
git config --list --show-origin
```

5. Review the output and note that settings come from different files: the system file in the Git installation folder and the global **.gitconfig** file in the user home folder. Press **q** if the output is displayed one page at a time.

Task 4: Verify the GitHub authentication method

1. In **Git Bash**, run the following command:

   ```
   git config --show-origin --get-all credential.helper
   ```

2. Verify that the output includes `manager`. This value indicates that **Git Credential Manager** handles authentication. The first time Git connects to GitHub, it opens a browser window to sign in and then stores the credential securely.

   Note: GitHub does not accept account passwords for Git operations. Git Credential Manager is included with Git for Windows. On macOS and Linux, install it with the following commands, or install the GitHub CLI and run `gh auth login` instead:

   ```
   # macOS
   brew install --cask git-credential-manager
   # Linux: download the .deb package from the GCM releases page, then run
   sudo dpkg -i <gcm-package>.deb
   git-credential-manager configure
   ```

   The Git Credential Manager packages are available at https://github.com/git-ecosystem/git-credential-manager/releases.

Exercise 2: Work with a local repository by using the Git command line

In this exercise, a new repository is created on the local computer, and changes are recorded, reviewed, undone, and merged without any connection to GitHub.

The exercise consists of the following tasks:

- Task 1: Create a local repository
- Task 2: Add files and create the first commit
- Task 3: Review and undo changes
- Task 4: Ignore files
- Task 5: Create a branch and merge it

Task 1: Create a local repository

1. In **Git Bash**, run the following commands to create a working folder for the labs and a folder for the new repository:

```
mkdir -p ~/labs/DevOpsGitLabRepo
cd ~/labs/DevOpsGitLabRepo
```

2. Run the following command to turn the folder into a Git repository:

```
git init
```

3. Note the message **Initialized empty Git repository**. Run `ls -a` and note the hidden **.git** folder. This folder stores the complete history of the repository.
4. Run the following command and review the output:

```
git status
```

5. Note that the current branch is **main** and that there are no commits yet.

Task 2: Add files and create the first commit

1. Run the following commands to create a **README.md** file:

```
echo "# DevOps Git Lab" > README.md
echo "This repository is used to practice Git commands." >> README.md
```

2. Run the following command to create a simple **index.html** page. Copy and paste all the lines at once.

```
cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
  <head><title>DevOps Git Lab</title></head>
  <body><h1>Welcome to the DevOps Git Lab</h1></body>
</html>
EOF
```

3. Run `git status`. Note that both files are listed as **Untracked files**. Git detects the files but does not track them yet.
4. Run the following commands to stage the **README.md** file and review the status again:

```
git add README.md
git status
```

5. Note that **README.md** appears under **Changes to be committed** (the staging area), while **index.html** is still untracked.
6. Run the following commands to stage all remaining files and create the first commit:

```
git add .
git commit -m "Add README and home page"
```

7. Run the following commands to review the history:

```
git log
git log --oneline
```

8. Note the commit hash, the author, the date, and the message. The `--oneline` option shows an abbreviated hash and the message only.

   Note: When the output of a command such as `git log` is longer than the window, Git displays it one page at a time. Press **Space** to scroll and **q** to return to the prompt.

Task 3: Review and undo changes

1. Run the following command to add a line to **README.md**:

   ```
   echo "Maintained by the DevOps team." >> README.md
   ```

2. Run `git diff`. The output shows the added line with a plus sign (**+**). These changes are in the working directory and are not staged.
3. Run the following commands to stage the change and compare the outputs of the two `git diff` commands:

   ```
   git add README.md
   git diff
   git diff --staged
   ```

4. Note that `git diff` now returns no output, and `git diff --staged` shows the change that will be included in the next commit.
5. Run the following command to commit the change:

   ```
   git commit -m "Add maintainer note to README"
   ```

6. Run the following commands to simulate an unwanted edit to **index.html** and review the status:

   ```
   echo "temporary text" >> index.html
   git status
   ```

7. Run the following commands to discard the edit and confirm that the file returned to its last committed version:

```
git restore index.html
git status
cat index.html
```

   Important: `git restore <file>` permanently discards uncommitted changes in that file.

8. Run the following commands to create a notes file, stage it, and then remove it from the staging area:

```
echo "Draft ideas" > notes.txt
git add notes.txt
git restore --staged notes.txt
git status
```

9. Note that **notes.txt** is listed as untracked again. `git restore --staged` removes a file from the staging area and keeps its content.

Task 4: Ignore files

1. Run the following commands to create a **.gitignore** file that excludes the notes file and any log files:

```
echo "notes.txt" > .gitignore
echo "*.log" >> .gitignore
git status
```

2. Note that **notes.txt** no longer appears in the output, and **.gitignore** is listed as a new untracked file.
3. Run the following commands to commit the **.gitignore** file:

```
git add .gitignore
git commit -m "Add .gitignore"
```

Task 5: Create a branch and merge it

1. Run the following commands to create a branch named **feature/about-page**, switch to it, and list the local branches:

```
git switch -c feature/about-page
git branch
```

2. Note that an asterisk marks the current branch.
3. Run the following commands to add an about page on the new branch:

```
echo "<h1>About the DevOps Git Lab</h1>" > about.html
git add about.html
git commit -m "Add about page"
```

4. Run the following commands to switch back to the **main** branch and list the files:

```
git switch main
ls
```

5. Note that **about.html** is not present. The file exists only on the **feature/about-page** branch.
6. Run the following commands to merge the branch into **main** and review the history:

```
git merge feature/about-page
git log --oneline --graph --all
```

7. Note the message **Fast-forward**. Because **main** had no new commits since the branch was created, Git moved **main** forward to the latest commit of the branch. When both branches contain new commits, Git creates a merge commit instead.
8. Run the following command to delete the merged branch:

```
git branch -d feature/about-page
```

Exercise 3: Synchronize the local repository with GitHub

In this exercise, the local repository is published to a new GitHub repository, and changes are synchronized in both directions.

The exercise consists of the following tasks:

- Task 1: Create an empty GitHub repository
- Task 2: Connect the local repository and push the commits
- Task 3: Fetch and pull changes made on GitHub
- Task 4: Publish a branch and merge it with a pull request
- Task 5: Tag a release and push the tag

Task 1: Create an empty GitHub repository

1. In the web browser, navigate to GitHub and sign in with the GitHub account used in Lab 01A.
2. Select the plus (**+**) icon in the upper right corner of the page and then select **New repository**.
3. On the **Create a new repository** page, do the following actions:
    - In the **Owner** drop-down list, select the GitHub user account name.
    - In the **Repository name** text box, enter `DevOpsGitLabRepo`.
    - Change the visibility of the repo to **Private**.
    - Do not add a README file, a **.gitignore** file, or a license. The repository must be empty because the content comes from the local repository.
4. Select **Create repository**.
5. On the **Quick setup** section of the new repository page, ensure that **HTTPS** is selected and select the copy icon next to the repository URL. The URL has the format `https://github.com/<github-username>/DevOpsGitLabRepo.git`.

Task 2: Connect the local repository and push the commits

1. Switch to **Git Bash** and ensure that the current folder is **~/labs/DevOpsGitLabRepo**.

2. Run the following command to add a remote named **origin**. Replace `<repository-url>` with the URL copied in the previous task. To paste in Git Bash, right-click the window and select **Paste**, or press **Shift+Insert**.

```
git remote add origin <repository-url>
git remote -v
```

3. Run the following command to push the **main** branch to GitHub and set it as the upstream branch:

```
git push -u origin main
```

4. When the **Connect to GitHub** window of Git Credential Manager appears, select **Sign in with your browser**, complete the sign-in in the browser, and authorize **Git Credential Manager**. Then return to **Git Bash**.
5. Review the output and note the line **branch 'main' set up to track remote branch 'main' from 'origin'** (The output can be slightly different). From now on, `git push` and `git pull` on **main** work without additional arguments.
6. Switch to the web browser and refresh the **DevOpsGitLabRepo** page. Verify that the **README.md**, **index.html**, **about.html**, and **.gitignore** files appear, and select the commits link to review the history that was created locally.

Task 3: Fetch and pull changes made on GitHub

1. On the **DevOpsGitLabRepo** page in GitHub, select **README.md** and then select the pencil icon (**Edit this file**).
2. Add the following line at the end of the file:

```
Updated from the GitHub web editor.
```

3. Select **Commit changes...** and, in the **Commit changes** window, enter `Update README from GitHub` as the commit message, keep **Commit directly to the main branch** selected, and select **Commit changes**.
4. Switch to **Git Bash** and run `git status`. Note that Git still reports that the branch is up to date. Git does not contact GitHub until a command such as `git fetch` runs.

5. Run the following commands to download the new commit without changing the local files:

```
git fetch
git status
```

6. Note the message **Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded**.
7. Run the following commands to review the incoming commit and its changes before integrating them:

```
git log --oneline main..origin/main
git diff main origin/main
```

8. Run the following commands to integrate the remote changes and review the file:

```
git pull
cat README.md
```

9. Note that the line added on GitHub is now present in the local copy. `git pull` runs `git fetch` followed by `git merge`.

Task 4: Publish a branch and merge it with a pull request

1. In **Git Bash**, run the following commands to create a branch with a new contact page:

```
git switch -c feature/contact-page
echo "<h1>Contact the DevOps team</h1>" > contact.html
git add contact.html
git commit -m "Add contact page"
```

2. Run the following command to publish the branch to GitHub:

```
git push -u origin feature/contact-page
```

3. Note that the output includes a link to create a pull request for the new branch.

4. Switch to the web browser and open the **DevOpsGitLabRepo** page. In the banner that reports the recent push of **feature/contact-page**, select **Compare & pull request**.
5. On the **Open a pull request** page, ensure that the base branch is **main** and the compare branch is **feature/contact-page**, keep the title **Add contact page**, and select **Create pull request**.
6. Select the **Files changed** tab and review the changes. Then return to the **Conversation** tab.
7. Select **Merge pull request**, select **Confirm merge**, and then select **Delete branch**.
8. Switch to **Git Bash** and run the following commands to update the local **main** branch and review the history:

```
git switch main
git pull
git log --oneline --graph -5
```

9. Note the merge commit with the message **Merge pull request #1** that GitHub created when the pull request was merged.
10. Run the following commands to delete the local branch, remove the reference to the deleted remote branch, and list all branches:

```
git branch -d feature/contact-page
git fetch --prune
git branch -a
```

11. Verify that only **main** and **remotes/origin/main** remain (with **remotes/origin/HEAD** if present).

Task 5: Tag a release and push the tag

1. In **Git Bash**, run the following commands to create an annotated tag for the current commit and review it:

```
git tag -a v1.0 -m "First release of the DevOps Git Lab site"
git tag
git show v1.0 --stat
```

2. Run the following command to publish the tag. Tags are not included in a regular `git push`.

```
git push origin v1.0
```

3. Switch to the web browser, refresh the **DevOpsGitLabRepo** page, select the branch drop-down list, and select the **Tags** tab. Verify that **v1.0** appears.

Exercise 4: Work with Git and GitHub in Visual Studio Code

In this exercise, the **DevOpsCoreIntroRepo** repository from Lab 01A is cloned and updated by using the **Source Control** view of Visual Studio Code, including a pull request that closes an issue and the resolution of a merge conflict.

The exercise consists of the following tasks:

- Task 1: Clone a GitHub repository in Visual Studio Code
- Task 2: Commit and synchronize changes
- Task 3: Create a branch, publish it, and close an issue with a pull request
- Task 4: Resolve a merge conflict
- Task 5: Review the history

Task 1: Clone a GitHub repository in Visual Studio Code

1. Start **Visual Studio Code**. If a folder is open, select **File** > **Close Folder**.
2. Select the **Source Control** icon in the Activity Bar, or press **Ctrl+Shift+G**, and then select **Clone Repository**.
3. Select **Clone from GitHub**. When prompted that the **GitHub** extension wants to sign in, select **Allow**, complete the sign-in in the browser, and return to Visual Studio Code.
4. In the list of repositories, select **<github-username>/DevOpsCoreIntroRepo**.
5. In the folder dialog, browse to the **labs** folder in the user home folder (for example, **C:\Users\<user-name>\labs**) and select **Select as Repository Destination**.
6. When prompted to open the cloned repository, select **Open**. If the **Do you trust the authors of the files in this folder?** dialog appears, select **Yes, I trust the authors**.
7. In the **Explorer** view, note the **README.md**, **LICENSE**, and **.gitignore** files created in Lab 01A. In the Status Bar at the bottom of the window, note the current branch, **main**.

Task 2: Commit and synchronize changes

1. In the **Explorer** view, open **README.md** and add the following line at the end of the file. Then press **Ctrl+S** to save the file.

   ```
   This repository is maintained with Visual Studio Code and Git.
   ```

2. Open the **Source Control** view. Note that **README.md** appears in the **Changes** section with the letter **M** (modified).
3. Select **README.md** in the **Changes** section to open the diff editor, which displays the original version and the modified version side by side.
4. Hover over **README.md** in the **Changes** section and select the plus (**+**) icon (**Stage Changes**). Note that the file moves to the **Staged Changes** section, which corresponds to `git add`.
5. In the commit message box at the top of the **Source Control** view, enter `Update README from Visual Studio Code` and select **Commit**.
6. Select **Sync Changes**. If a dialog explains that the action pulls and pushes commits from and to **origin/main**, select **OK**.
7. Switch to the web browser, open the **DevOpsCoreIntroRepo** repository, and verify that the README now includes the new line.

   Note: When GitHub Copilot is enabled, the sparkle icon in the commit message box generates a commit message from the staged changes. Always review a generated message before committing.

Task 3: Create a branch, publish it, and close an issue with a pull request

1. In the web browser, in the **DevOpsCoreIntroRepo** repository, select the **Issues** tab, open the **Install GitHub on Pilot Computers** issue created in Lab 01A, and note its number (for example, **#4**).

   Note: If the issue does not exist, select **New issue**, enter `Install GitHub on Pilot Computers` as the title, select **Create**, and note the number of the new issue.

2. Switch to Visual Studio Code. In the Status Bar, select the branch name **main**, select **Create new branch...**, enter `docs/pilot-setup`, and press **Enter**. Note that the Status Bar now shows **docs/pilot-setup**.
3. In the **Explorer** view, select the **New File** icon, enter `PILOT-SETUP.md` as the file name, and add the following content. Then save the file.

```
# Pilot computer setup

1. Install Visual Studio Code.
2. Install Git and configure user.name and user.email.
3. Clone DevOpsCoreIntroRepo and verify access with a pull.
```

4. In the **Source Control** view, hover over the **Changes** header and select the plus (**+**) icon to stage all changes. Enter `Add pilot computer setup guide` as the commit message and select **Commit**.
5. Select **Publish Branch** to push the branch to GitHub and set its upstream branch. This action corresponds to `git push -u origin docs/pilot-setup`.
6. Switch to the web browser and open the **DevOpsCoreIntroRepo** page. In the banner for **docs/pilot-setup**, select **Compare & pull request**.
7. In the description of the pull request, enter `Closes #<issue-number>`, replacing `<issue-number>` with the number noted in the first step, and select **Create pull request**.
8. Note that the issue now appears in the **Development** section of the pull request. Note: If it does not appear, add it manually.
9. Select **Merge pull request**, select **Confirm merge**, and then select **Delete branch**.
10. Select the **Issues** tab and verify that the **Install GitHub on Pilot Computers** issue is closed. Optionally, open the **DevOps Core Intro Project** board from Lab 01A and verify that the item moved to the **Done** column.
11. Switch to Visual Studio Code. In the Status Bar, select **docs/pilot-setup** and then select **main** to switch branches.
12. In the **Source Control** view, select **More Actions** (**...**) > **Pull**. Verify that **PILOT-SETUP.md** now appears in the **Explorer** view on the **main** branch.
13. Open the Command Palette (**Ctrl+Shift+P**), run **Git: Delete Branch...**, and select **docs/pilot-setup** to delete the local branch.

Task 4: Resolve a merge conflict

A merge conflict occurs when the same lines of a file change in two places. In this task, the first line of **README.md** is changed both on GitHub and in Visual Studio Code.

1. In the web browser, in the **DevOpsCoreIntroRepo** repository, open **README.md** and select the pencil icon (**Edit this file**).
2. Replace the first line of the file with the following text:

```
# DevOps Core Intro Repository (updated on GitHub)
```

3. Select **Commit changes...**, keep **Commit directly to the main branch** selected, and select **Commit changes**.

4. Switch to Visual Studio Code without synchronizing. Open **README.md**, replace the first line with the following text, and save the file:

   ```
   # DevOps Core Intro Repository (updated in Visual Studio Code)
   ```

5. In the **Source Control** view, stage the change, enter `Update README title in Visual Studio Code` as the commit message, and select **Commit**.
6. Select **Sync Changes**. Visual Studio Code pulls the commit from GitHub first, and Git reports a conflict. Note that **README.md** appears in the **Merge Changes** section of the **Source Control** view.
7. Open **README.md** from the **Merge Changes** section and review the conflict markers:
   - The lines between `<<<<<<< HEAD` and `=======` contain the **Current Change** (the local commit).
   - The lines between `=======` and `>>>>>>>` contain the **Incoming Change** (the commit from GitHub).
   - The actions **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**, and **Compare Changes** appear above the conflict.
8. Select **Resolve in Merge Editor**. Review the **Incoming** pane, the **Current** pane, and the **Result** pane at the bottom.
9. In the **Result** pane, replace the conflicting heading with the following text, so that the resolved file contains a single first line:

   ```
   # DevOps Core Intro Repository (updated on GitHub and in Visual Studio Code)
   ```

10. Select **Complete Merge**. Note that **README.md** moves to the **Staged Changes** section and that the commit message box contains a merge message.
11. Select **Commit** to create the merge commit, and then select **Sync Changes** to push the result to GitHub.
12. Switch to the web browser, refresh the **DevOpsCoreIntroRepo** page, and verify that the README displays the resolved heading.

    Note: In the command line, the same conflict is resolved by editing the file to remove the conflict markers, then running `git add README.md` and `git commit`.

Task 5: Review the history

1. In the **Source Control** view, expand the **Source Control Graph** section. Review the commits, including the merge commit from the pull request and the merge

commit created during the conflict resolution. Select a commit to list the files that it changed.

2. In the **Explorer** view, select **README.md** and expand the **Timeline** section at the bottom of the view. Note the list of commits that changed the file.
3. Select **Terminal** > **New Terminal** and run the following command in the integrated terminal:

```
git log --oneline --graph -8
```

4. Note that the command line shows the same history as the **Source Control Graph**. Visual Studio Code and the Git command line work on the same repository.

Review

The following table summarizes the operations practiced in this lab and their equivalents in the Git command line and in Visual Studio Code.

Operation	Git command line	Visual Studio Code
Create or clone a repository	`git init` `git clone <url>`	**Initialize Repository** **Clone Repository**
Review changes	`git status` `git diff`	**Source Control** view and diff editor
Stage changes	`git add <file>`	**Stage Changes** (+)
Commit	`git commit -m "<message>"`	Commit message box and **Commit**
Create and switch branches	`git switch -c <branch>` `git switch <branch>`	Branch name in the Status Bar
Publish a branch	`git push -u origin <branch>`	**Publish Branch**

Synchronize	`git fetch`
	`git pull`
	`git push`
		Sync Changes, **Pull**, **Push**

Resolve a conflict	Edit the file, `git add`, `git commit`	Merge editor, **Complete Merge**, **Commit**
Review the history	`git log --oneline --graph`	**Source Control Graph** and **Timeline**

The **DevOpsGitLabRepo** and **DevOpsCoreIntroRepo** repositories can be kept for reference. Later labs use the same GitHub account.